

Технологии глубокого обучения и нейронных сетей

Лекция 1

От монетки до языковой модели

О чем этот курс

Курс о больших языковых моделях: от принципа, на котором они обучаются, до работающего сервиса.

- | | | | |
|---|-----------------------------------|---|------------------------|
| 1 | От монетки до языковой модели | 5 | LLM в приложениях |
| 2 | Устройство трансформера | 6 | LLM в продакшне |
| 3 | Обучение языковых моделей | 7 | Передний край и синтез |
| 4 | Как модель становится ассистентом | | |

Семь лекций, одна идея: модель учит распределение, меняется только масштаб. Первые четыре про то, как это работает, оставшиеся о том, как это использовать.

Как устроен курс

Лекции

7 лекций по 2 академических часа с живыми демо.

Лабораторные

7 работ, 28 часов.

Ноутбуки на Kaggle или локально, сдача через личный репозиторий на GitHub.

Контроль

Курсовая: LLM-сервис из нескольких модулей.

Экзамен по билету из трех вопросов.

Бонус

«Охота за поломками».

Применение LLM в курсе про LLM

Три правила

1

Вы отвечаете за каждую строку

На защите можете объяснить и изменить свой код без ассистента.

2

Перепроверяйте модель

Она ошибается также уверенно, как и дает верные ответы.

3

Не теряйте свой проект

ИИ — это инструмент, а не замена.

Пример контроля с предыдущего слайда

Третье — про то, как я работал. Дважды я едва не сломал то, что не проверил: почти внёс в CI шаг, который на каждом деплое проставлял бы «Смартфоны» новым позициям каталога, — потому что поверил документации, утверждавшей идемпотентность миграций. И увёл нас в долгий разговор про объединение цен с характеристиками на посылке, которую сам же придумал. Оба раза остановились не потому, что я перепроверил, а потому что вы спросили. Закономерность неприятная: я надёжен там, где сверяю с фактом, и ненадёжен там, где рассуждаю по аналогии.

Фронтирная модель Claude Opus 5

Что такое машинное обучение

Данные → модель → предсказание на новых данных

С учителем

Есть правильные ответы. Регрессия предсказывает число, классификация предсказывает класс.

Без учителя

Ответов нет, ищем структуру: кластеризация, снижение размерности.

С подкреплением

Есть среда и награда. Агент учится действовать.

Генерация текста тоже задача с учителем. Учитель это сам текст: следующее слово всегда известно.

Данные

Модель знает только то, что было в данных

Мало данных: модель угадывает

Десять примеров это не закономерность, а совпадение. Уверенность растет вместе с числом примеров.

Не те данные: модель уверенно ошибается

Учили на летних фото, показали зимнее. Модель не знает, что чего-то не знает, и отвечает с той же уверенностью.

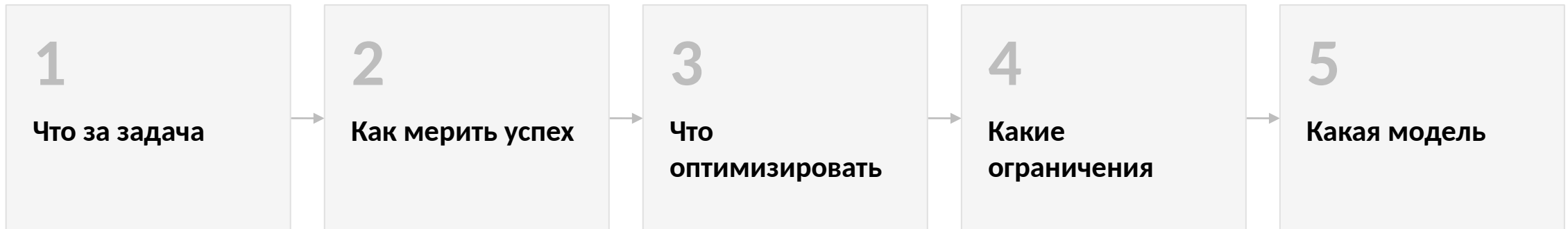
Старые данные: модель устаревает

Мир меняется, а данные, на которых учили, остаются прежними. Модель не сломалась, изменился мир.

Половина инженерной работы это данные, а не модель. На следующем слайде модель стоит последней.

Цикл разработки модели

Порядок важен. Модель выбирается последней.



Самая частая ошибка новичка: выбрать модель первой, а потом подгонять под нее задачу. В курсовой это будет заметно.

Как модель понимает, что ошиблась

MSE

Для чисел.

$$(y - \hat{y})^2$$

MAE

Для чисел с выбросами.

$$|y - \hat{y}|$$

Кросс-энтропия

Для классов.

$$-\log \hat{p}$$

Функция потерь это число: чем больше, тем хуже.

Обучение это подбор параметров, при которых это число минимально. Больше в обучении ничего нет.

Почему именно квадрат и логарифм: заметка [why-loss.md](#) в репозитории, для любознательных.

Правила, написанные руками

СПАМ-ФИЛЬТР НА ПРАВИЛАХ

если «выигрыш» и «срочно» → спам

если отправитель в адресной книге → не спам

если ссылок больше трех → спам

если тема заглавными буквами → спам

...

правило 137: ???

1

Работает, пока правил десятки.

2

Ломается, когда мир меняется.

3

Никто не помнит, зачем правило 137.

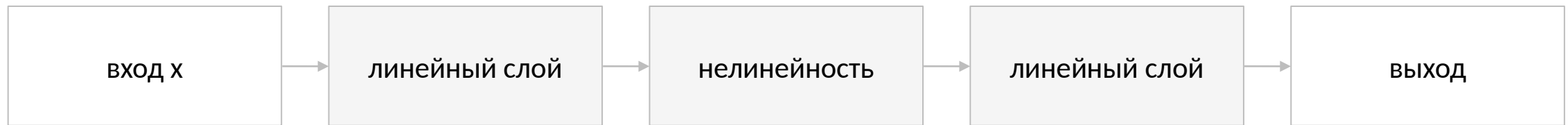
Первый шаг: не писать правило, а подобрать его по примерам.

Классическое ML: человек делает признаки, модель подбирает веса

СЕМЕЙСТВО	ИДЕЯ	КОГДА РАБОТАЕТ
Линейные модели: регрессия, логрег, SVM	Прямая или плоскость: разделяет классы или описывает зависимость	Много признаков, мало данных, нужна объяснимость
Ближайшие соседи	Похожие объекты дают похожие ответы, учиться не нужно	Мало данных, хорошая мера схожести
Деревья и ансамбли: случайный лес, бустинг	Иерархия вопросов; сотни деревьев голосуют или исправляют друг друга	Таблицы: бустинг до сих пор бьет нейросети, лаба 2
Наивный Байес	Счетчики слов по классам, вероятности перемножаются	Спам-фильтр за вечер, быстрый бейзлайн для текста
Кластеризация, PCA	Структура без разметки, сжатие размерности	Разведка данных; отсюда растут эмбединги

Общее у всех: какие признаки подать, решает человек. Как их взвесить, подбирает модель.

Нейросети: пусть признаки выучиваются сами



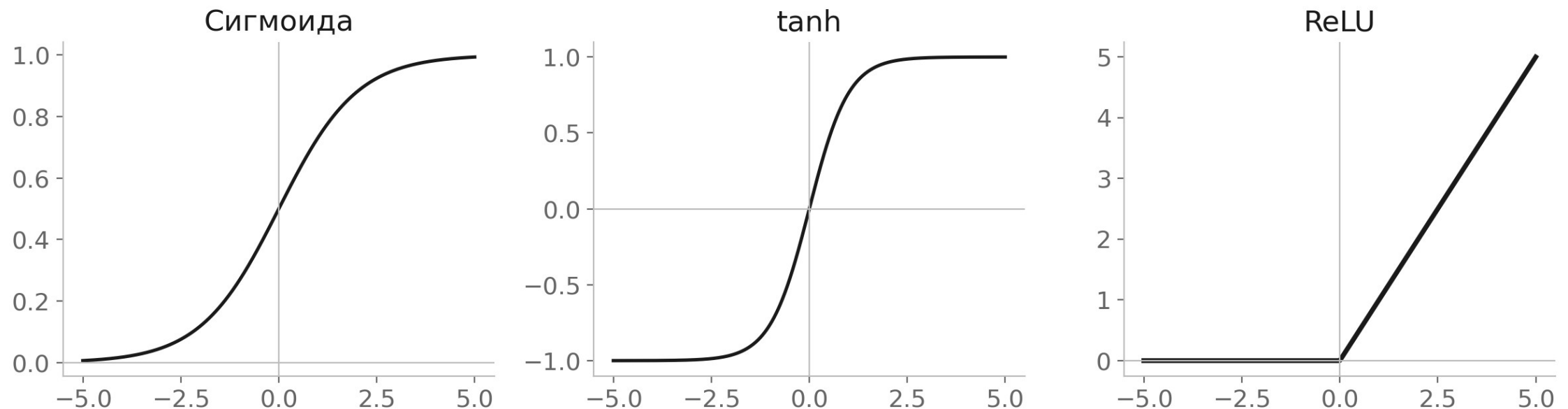
$$f(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Каждый слой это линейная модель с прошлого слайда, между слоями нелинейность. Первые слои собирают простые признаки из сырых данных, следующие собирают сложные из простых.

Отсюда «глубокое»: слоев много. Никто эти признаки не программировал.

Функции активации

Нелинейность между слоями. Без нее композиция линейных слоев остается линейной.



Сигмоида и tanh это история: хороши для объяснения, плохи для обучения глубоких сетей, градиент через них затухает. ReLU: ноль слева, прямая справа, одна операция, градиент не затухает. Она и ее варианты это сегодняшний день.

Что изменилось технически

1

Аналитического решения нет

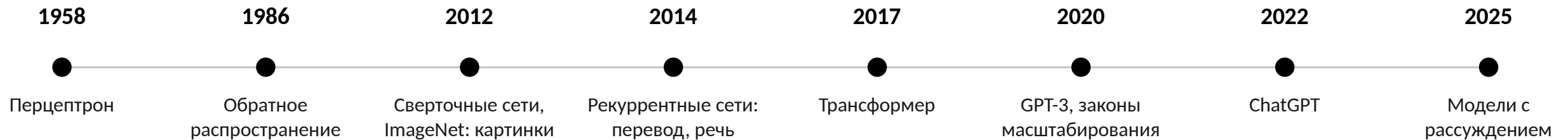
Параметров миллионы, формулы для ответа нет. Максимум ищется шагами: подобрали параметры чуть лучше, еще чуть лучше, и так миллионы раз. Это градиентный спуск.

2

Производную считает автоматика

Обратное распространение ошибки. Это делает фреймворк; как именно, разбор для любознательных в README.

От перцептрона до языковой модели



Пятьдесят лет идея ждала данных и вычислений. Потом за десять лет: картинки, звук, текст.

Одна архитектура закрывает почти все задачи с текстом. Это предмет курса.

Где читать

Все ссылки в README репозитория курса

- Курс Е. Соколова (ml-course-hse) и Яндекс ML Book: начинать отсюда, ближе к практике
- Курсы К. В. Воронцова и А. Г. Дьяконова: когда захочется строгости
- Deep Learning School (DLS): нейросети с нуля
- В. Бабушкин, «Machine Learning System Design»: для курсовой
- Ф. Шолле, «Глубокое обучение на Python»: от тензора до своей GPT
- Kaggle, Papers with Code: практика и передний край
- Документация scikit-learn, PyTorch, transformers



github.com/pmenshih/tgo-course

Монетка

100 бросков, 63 орла.

Какова вероятность орла?

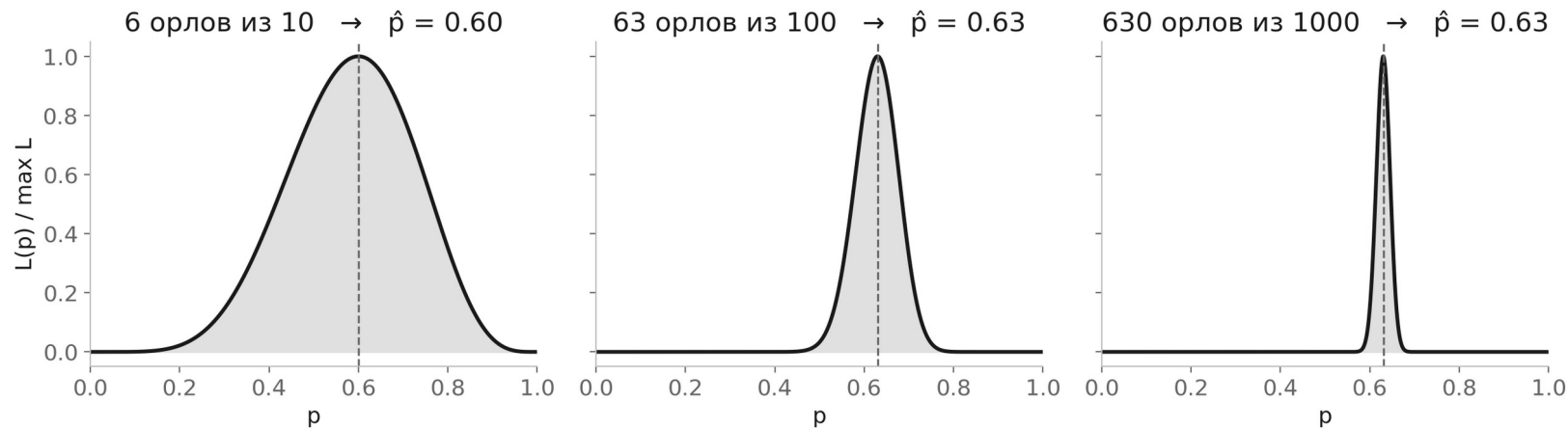
Ответ очевиден. Но откуда он взялся? Тот же вопрос стоит перед любой моделью.

0.63

Правдоподобие

$$L(p) = p^{63} (1 - p)^{37}$$

Насколько вероятны наши данные, 63 орла и 37 решек, при таком p .



Вершина говорит, какой ответ лучший.

Ширина говорит, насколько ему верить.

Больше данных, острее пик.

ПРИНЦИП

Выбираем параметр, при котором данные наиболее правдоподобны. Это и есть обучение.

Потеря это минус логарифм вероятности

потеря = $-\log P(\text{правильный ответ})$

Обучение делает вероятность правильных ответов как можно больше. Функция потерь это та же вероятность: с логарифмом, чтобы произведение стало суммой, и с минусом, чтобы минимизировать.

Ошибки разбросаны колоколом

Минус логарифм дает квадрат. Это MSE.

Ответ «да или нет»

Минус логарифм дает кросс-энтропию.

Три потери с девятого слайда это одно правило при разных предположениях о данных. Вывод: [why-loss.md](#).

Модель — семейство распределений.

Обучение — выбор члена семейства, при котором данные наиболее правдоподобны.

Дальше меняется только богатство семейства и масштаб данных.

Все, что мы будем изучать, от бустинга до GPT, укладывается в эти три предложения.

Доля это оценка

Монетка

63 орла из 100

$$p = 0.63$$

Слово

после «я» 120 раз из 1000 шло «хочу»

$$P(\text{хочу} \mid \text{я}) = 0.12$$

Вероятность следующего слова это доля. Своя монетка на каждое слово-предшественник, и сторон у нее столько, сколько слов.

Языковая модель из счетчиков

Таблица биграмм

я → хочу	0.12
я → не	0.09
я → думаю	0.07
я → буду	0.05
я → знаю	0.04
...	

Генерация

я → хочу → сказать → что → ...

Каждое следующее слово выбирается по долям из строки предыдущего. Получается связный на вид бред.

Пары не было

Вероятность ноль. Лечится сглаживанием.

Смысла нет

Модель помнит одно слово. Лечится длинным контекстом.

Лабораторная 1: вы соберете такую модель без библиотек. Большая модель делает то же, только предшественников тысячи.

Языковая модель вживую

1

Распределение следующего токена

«import numpy as» против «Сегодня я хочу»: один столбик почти в единицу против размазанного распределения. Та же таблица, что на слайде 22, только предшественников тысячи.

2

Температура

Одни и те же выходы модели при $T = 0.5, 1, 2$. Знаний не прибавилось, изменилась острота.

3

Правдоподобие текста

Связное предложение против тех же слов вперемешку. Сумма логарифмов у связного выше: вот что максимизировалось при обучении.

Правдоподобие: монетка и языковая модель

МОНЕТКА

$$\log L = 63 \log p + 37 \log(1 - p)$$

Один параметр, сто бросков

ЯЗЫКОВАЯ МОДЕЛЬ

$$\log L = \sum_{t=1}^T \log P(x_t \mid x_{<t})$$

Миллиарды параметров, триллионы токенов

Обучение языковой модели это подбор миллиардов параметров, при которых тексты корпуса наиболее правдоподобны. Вероятности, логарифм, сумма, максимум. То же, что мы делали с монеткой.

Уверенная чушь

ПРОМПТ

Приведи пример использования `torch.nn.functional.smooth_relu` и объясни параметр `beta`.

ОТВЕТ МОДЕЛИ · Qwen3-0.6B, температура 0.7

Приведем пример использования `torch.nn.functional.smooth_relu` и объясню параметр `beta`.

Пример использования:

```
import torch
import torch.nn.functional as F

# Создаем входные данные
x = torch.randn(100, 100) # 100x100 tensor

# Применяем функцию smooth_relu
output = F.smooth_relu(x, beta=0.1)

print(output)
```

Объяснение:

`F.smooth_relu(x, beta=0.1)` — это функция, которая применяет ReLU к входному тензору `x`, но с смягчением параметра `beta`, что уменьшает "ненужные" изменения по сравнению с обычным ReLU.

[...]

Функции `torch.nn.functional.smooth_relu` не существует. Сигнатура, параметр и пример придуманы.

СЛУЧАЙ

Спросил модель про функцию библиотеки, которой не существует. Получил сигнатуру, описание параметра и пример кода.

В ЧЕМ ОШИБКА

Факта нет. Функции нет. Все придумано, и выглядит безупречно.

МЕХАНИЗМ

Модель обучена максимизировать правдоподобие текста, а не истинность утверждений. Правдоподобный текст про несуществующую функцию это ровно то, что оптимизировалось. Уверенный тон не признак знания, это свойство вероятной последовательности токенов.

Главный вывод

Галлюцинация не баг реализации, а следствие целевой функции.

«Улучшением модели» она не чинится. Чинится снаружи: поиск по источникам, верификация, тесты.

О том как именно поговорим в лекциях про приложения и продакшн.

Охота за поломками

Бонус к экзамену. Галлюцинация уже засчитана, ищите другие.

1

Найдите воспроизводимую поломку

Соглашательство под давлением, потеря инструкции на длинной задаче, инъекция промпта, арифметика.

2

Оформите протокол

Как воспроизвести и какой механизм из лекций объясняет. Шаблон в репозитории: `cases/protocol_template.md`.

3

Лучшие разбираем на лекциях

В этой же рубрике «Вскрытие».

Лабораторная 1: языковая модель из счетчиков

ТЗ в репозитории, папка labs/01. Корпус берете сами.

Своя реализация

Без библиотек: разбить корпус на слова, посчитать биграммы, вероятность как доля, генерация, температура, сглаживание.

Проверка

Правдоподобие связного предложения против перемешанного. Качество на отложенном тексте.

Бонус

Триграммы. Сравнение с Qwen на тех же предложениях. Наивный Байес на тех же счетчиках.

Спасибо

СЛЕДУЮЩАЯ ЛЕКЦИЯ

Устройство трансформера: как слово становится вектором, почему рекуррентные сети не справились и что такое внимание.

github.com/pmenshih/tgo-course

Материалы, ноутбуки демонстраций, задания лабораторных, шаблон протокола поломки

